Written assignment-1

Q1) What are the key features and advantages of AWS Cloud 9 ? LO1

AWS Cloud9 – Key Features and Advantages

1. Cloud-Based IDE

AWS Cloud9 is a browser-accessible development environment that eliminates the need for local setup. It includes a code editor, terminal, and debugger—all integrated into one interface. Developers can start coding instantly without installing tools or configuring environments.

2. Real-Time Collaboration

Multiple users can work on the same codebase simultaneously. Cloud9 supports shared environments where teammates can see each other's cursors, chat live, and co-edit files. This is especially useful for remote teams, pair programming, and mentoring.

3. Built-In Terminal with AWS CLI

The IDE includes a Linux-based terminal with pre-installed AWS CLI. Developers can run shell commands, manage AWS services, and deploy applications directly from the IDE—without switching to another tool or terminal.

4. Preconfigured Development Environments

Cloud9 supports multiple programming languages like JavaScript, Python, PHP, Ruby, Go, and C++. It comes with essential tools like Git, Docker, and Node.js pre-installed. This reduces setup time and ensures consistency across teams.

5. Deep Integration with AWS Services

Cloud9 is tightly integrated with AWS. Developers can create and manage EC2 instances, Lambda functions, S3 buckets, and DynamoDB tables directly from the IDE. It's ideal for building serverless applications and cloud-native solutions.

6. Secure Access via IAM

Access to Cloud9 environments is controlled using AWS Identity and Access Management (IAM). This ensures secure, role-based access without exposing credentials. Developers can work safely within defined permission boundaries.

7. Environment Portability and Management

Cloud9 environments can be EC2-backed or connected via SSH to existing servers. Developers can clone, pause, or terminate environments as needed. This flexibility supports both cloud-based and hybrid workflows.

Advantages of AWS Cloud9

- **No local setup**: Code from any device with a browser.
- **Scalable**: Use AWS compute resources for heavy workloads.
- **Secure**: IAM-based access control and no credential leakage.
- **Collaborative**: Real-time editing and communication.
- **Integrated**: AWS CLI, Git, and language runtimes built-in.
- **Efficient**: Reduces context switching and setup overhead.

Q2) What is the difference between Cloud 9 and Lambda ? LO1,LO6

The difference between cloud9 and Lambda is given:

Feature	**AWS Cloud9**	**AWS Lambda**
Type	Cloud-based Integrated Development Environment (IDE)	Serverless compute service
Purpose	Write, edit, and debug code in the cloud	Run code in response to events without managing servers
Usage	Development and collaboration	Execution of backend logic, automation, APIs, etc.
Environment	Full Linux-based dev environment with terminal access	Stateless function container managed by AWS
Language Support	Multiple: JavaScript, Python, PHP, Go, etc.	Multiple: Node.js, Python, Java, Go, .NET, Ruby
Execution Model	Manual execution via terminal or editor	Event-driven execution (e.g., API call, file upload)
Persistence	Persistent workspace and file system	Ephemeral—no persistent storage between executions
Scalability	Not auto-scaled; depends on EC2 instance	Automatically scales with incoming requests

Feature	AWS Cloud9	AWS Lambda
Pricing	Based on EC2 usage and storage	Pay-per-use (requests and compute time)
Ideal For	Coding, debugging, team collaboration	Microservices, automation, backend APIs

Q3) What are the three components of AWS Lambda ? LO1,LO6

AWS Lambda is characterized by being serverless, provisioningless, and function-based. Its main use is located in the computing layer of an application which does not have a server, in addition to its main purpose is to create applications based on events and that have the possibility of being activated by various AWS events.

If the situation of having multiple concurrent events arises, AWS lambda will trigger multiple copies of the function, making Lambda a Function of Service (FaaS) type.

In AWS Lambda, the Function is the core component where the actual code resides. This code is written to perform a specific task, such as processing an image, handling an API request, or responding to a database update. The function is stateless and designed to execute quickly and efficiently. Developers write this code in supported languages like Python, Node.js, Java, or Go, and it is uploaded to AWS either directly or via deployment tools. The function is the executable logic that AWS invokes when triggered by an event.

The Configuration component defines how the function should behave during execution. It includes settings such as memory allocation, timeout duration, environment variables, IAM roles for permissions, and concurrency limits. These parameters allow developers to fine-tune performance, security, and resource usage. For example, increasing memory can improve execution speed, while setting a timeout ensures that long-running functions are terminated appropriately. Configuration also determines how the function integrates with other AWS services and manages its runtime environment.

The Event Source is the trigger mechanism that initiates the function's execution. It can be any AWS service capable of generating events, such as S3 (file uploads), DynamoDB (table updates), API Gateway (HTTP requests), or CloudWatch (scheduled tasks). Third-party services or custom applications can also act as event sources via AWS SDKs or messaging systems. While event sources are optional, they are essential for building reactive, event-driven architectures. When configured, AWS automatically invokes the function whenever the specified event occurs, enabling seamless automation and integration across cloud services.

Q4) Discuss the difference between Kubernetes and other container orchestration tools like Docker Swarm? LO1,LO2

Architecture & Complexity

- Kubernetes has a more complex architecture with components like etcd, kube-apiserver, kube-scheduler, and kube-controller-manager. It offers fine-grained control over deployments, networking, and storage.
- Docker Swarm is simpler and tightly integrated with Docker. It uses a manager-worker model and is easier to set up for small-scale applications.

Scalability & Performance

- Kubernetes is designed for large-scale, production-grade workloads. It supports auto-scaling, rolling updates, and self-healing mechanisms.
- Docker Swarm is suitable for smaller clusters and simpler use cases. It lacks advanced scaling and recovery features found in Kubernetes.

Load Balancing & Networking

- Kubernetes uses a robust networking model with built-in service discovery, DNS-based load balancing, and support for ingress controllers.
- Docker Swarm provides basic internal load balancing and service discovery but lacks advanced ingress and traffic routing capabilities.

Configuration & Management

- Kubernetes uses YAML manifests for declarative configuration, offering extensive customization and automation.
- Docker Swarm uses Docker Compose files with limited orchestration features and less flexibility in defining complex deployments.

Security & Role Management

- Kubernetes supports Role-Based Access Control (RBAC), secrets management, and network policies.
- Docker Swarm has basic security features like mutual TLS between nodes but lacks granular access control.

Ecosystem & Community

- Kubernetes has a vast ecosystem with tools like Helm, Istio, Prometheus, and ArgoCD. It's backed by the Cloud Native Computing Foundation (CNCF) and widely adopted across industries.

- Docker Swarm has a smaller community and fewer integrations. Docker Inc. has shifted focus toward Kubernetes support.

Written Assignment 2

Q1)What are the key Terraform commands, explain ? LO1,LO3

The key terraform commands are as shown :

1.terraform init

This command initializes a Terraform working directory. It sets up the backend configuration, downloads required provider plugins, and prepares the environment for further Terraform operations. You should run terraform init once when starting a new project or after modifying the provider configuration in your .tf files.

2.terraform plan

This command shows you what Terraform will do before it actually makes any changes. It compares your current infrastructure state with the desired state defined in your configuration files and outputs a detailed execution plan. This helps you verify and review changes before applying them, making it a safe step in any deployment workflow.

3.terraform apply

This command executes the actions proposed in the plan and creates or updates infrastructure accordingly. It prompts for confirmation before proceeding unless you use the -auto-approve flag. After running terraform apply, your infrastructure will match the configuration defined in your .tf files.

4.terraform destroy

This command removes all infrastructure managed by your Terraform configuration. It's used when you want to tear down your environment completely. Like apply, it shows a plan and asks for confirmation before executing unless you use -auto-approve.

5.terraform validate

This command checks whether your Terraform configuration files are syntactically valid. It does not access remote services or check the correctness of resource arguments, but it ensures that your .tf files are properly structured and free of syntax errors.

6.terraform fmt

This command formats your Terraform configuration files to follow the standard style conventions. It helps maintain clean, readable code and is especially useful when working in teams or reviewing code in version control.

7.terraform state

This command lets you inspect and manage the Terraform state file, which tracks the current state of your infrastructure. You can use subcommands like terraform state list to view resources or terraform state rm to remove specific items from state tracking.

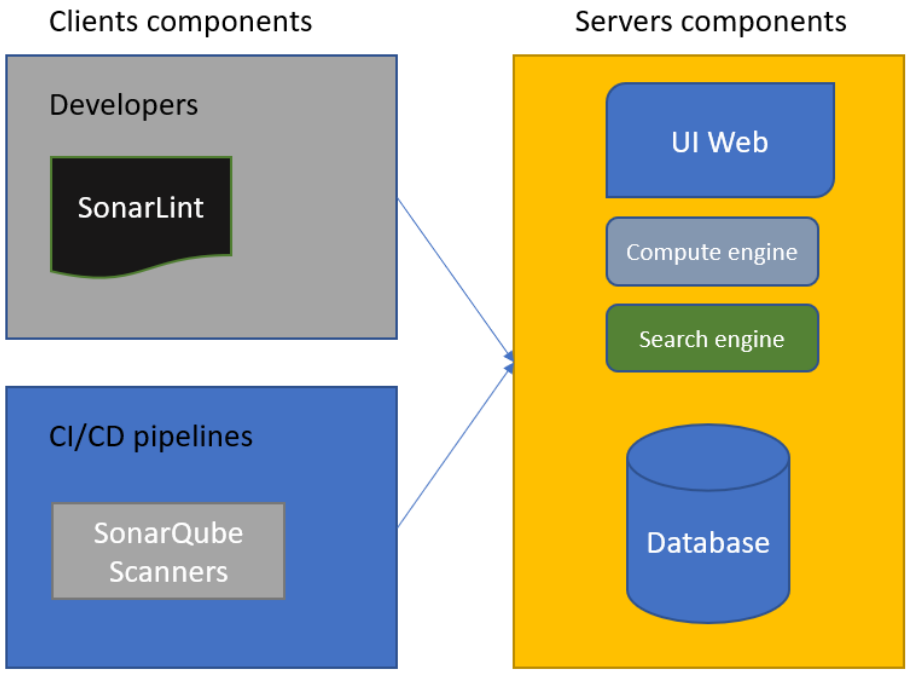
8.terraform output

This command displays the output values defined in your configuration after running terraform apply. These outputs can include IP addresses, resource IDs, or any other values you've marked as outputs in your .tf files. It's useful for retrieving dynamic values that other tools or scripts might need.

Q2)Explain the key components of Sonarqube's architecture? LO1,LO4

SonarQube is a client-server tool, which means that its architecture is composed of artifacts on the server side and also on the client side.

A simplified SonarQube architecture is shown in the following diagram:



Client components are primarily involved in the development and automated testing/deployment processes:

1.**Developers:** This area focuses on individual developer workstations or environments.

> **SonarLint:** This is an IDE (Integrated Development Environment) extension used by developers to provide real-time feedback on code quality and security issues while they write code. It acts as a local, "shift-left" tool for static analysis.

2.**CI/CD pipelines:** This represents the continuous integration and continuous delivery/deployment automation infrastructure.

> **SonarQube Scanners:** These are tools integrated into the CI/CD pipeline (e.g., Jenkins, GitHub Actions, GitLab CI) that perform static analysis on the entire codebase as part of the build process. They typically send their findings to a central SonarQube Server component for detailed reporting and quality gating.

Server components represent the core application or platform that processes requests and manages data:

- The entire server-side application is contained within the large yellow box.
- **UI Web:** This is the user interface, likely a web application, that users interact with.
- **Compute engine:** This component handles the business logic, processing, and computational tasks of the application.
- **Search engine:** This component is dedicated to indexing data and executing fast, complex search queries, often used by the UI Web or Compute engine.
- **Database:** This is the persistent storage layer for the application's data.

The client components (specifically the code/artifacts produced by Developers and analyzed by CI/CD pipelines) interact with and feed into the server environment, resulting in a process where developed and validated code is deployed to run the server component

Q3)What are the 3 full deployment modes that can be used for AWS? LO1,LO3

The three full deployment modes commonly used in AWS environments, each suited to different stages of development and operational needs are:

1.**Development Mode**

Development mode is used for building and testing applications in a non-production environment. Resources are typically provisioned with minimal cost and scale, allowing developers to iterate quickly. This mode often includes mock data, debugging tools, and relaxed security settings to facilitate experimentation and rapid prototyping.

2.**Staging Mode**

Staging mode replicates the production environment as closely as possible but is isolated from live users. It is used for final testing, performance validation, and quality assurance before deployment. This mode helps identify issues that may not surface during development and ensures that the application behaves as expected under realistic conditions.

3.**Production Mode**

Production mode is the live environment where real users interact with the application. Resources are provisioned for high availability, scalability, and security. Monitoring, logging, and backup systems are typically enabled, and infrastructure is optimized for performance and reliability. This mode represents the final and most critical stage of deployment.

Q4)What are the benefits of NAGIOS in AdvDevOps ? LO1,LO5

They key benefits of using of using Nagios are:

- **Real-time Monitoring**: Nagios provides continuous monitoring of servers, applications, services, and network devices, helping teams detect issues before they impact users.

- **Alerting and Notifications**: It sends alerts via email, SMS, or custom scripts when thresholds are breached or failures occur, enabling rapid incident response.

- **Scalability**: Nagios can monitor thousands of hosts and services using plugins and distributed architectures, making it suitable for large-scale DevOps environments.

- **Extensibility via Plugins**: With a rich ecosystem of community and custom plugins, Nagios can be tailored to monitor virtually any system metric or application.

- **Performance Graphing**: Integration with tools like PNP4Nagios or Grafana allows visualization of performance data over time, aiding in capacity planning and optimization.

- **Infrastructure Visibility**: Nagios offers a centralized dashboard that gives DevOps teams a clear view of system health, dependencies, and service status.

- **Automation Integration**: It can trigger automated remediation scripts or workflows when specific conditions are met, aligning with DevOps principles of self-healing systems.

- **Audit and Reporting**: Nagios logs all events and alerts, supporting compliance, post-mortem analysis, and continuous improvement.

- **Multi-platform Support**: It supports monitoring across Linux, Windows, Unix, and cloud platforms, making it versatile for hybrid infrastructure setups.

- **Community and Enterprise Editions**: Nagios Core is open-source and widely adopted, while Nagios XI offers enterprise-grade features for advanced DevOps teams.